GEN-AI

PART-1
LANGCHAIN, LLM & PROMPT ENGINEERING

→ **Generative AI (Gen-AI)**

It is a type of AI that can generate or create new content such as-

- text
- Images
- audio-video
- code
- Designs

By learning patterns from large amount of existing data.

↳ **Working**

(1.) Training : Learns patterns from large datasets like - Image, text, audio, video etc.

(2.) Prompt : Receives a user's input

(3.) Processing : Understands the prompt & converts it into tokens

(4.) Prediction: Predicts the next token repeatedly using the learned patterns.

(5.) Generative: combine the predicted tokens to produce a complete response.

→ Evolution of Gen-AI

the development can be divided into four layers.

↳ Layer: 1 - Research

Researchers publish new AI architectures

Ex-
- LSTM (Long short Term Memory)
- GRU (Gated Recurrent Unit)
- Transformer

* Famous paper
"Attention is all you need" (google brain - 2017) for Transformer architecture

↳ Layer: 2 - Foundation Models

Large companies train massive AI models using research

Ex-
- open AI → chatgpt
- Google → Gemini
- Antropic → claude
- Meta → Llama

- Training Requires
* Huge datasets
* Thousands of GPUs
* Massive computing power

↳ Layer: 3 - AI Providers

Providers (usually AI companies) host trained models & expose them through APIs.

Ex-
- open AI API
- Google AI studio
- Groq cloud
- Mistral AI
- Hugging Face

- Developers use API keys instead of training model themselves.

Layer : 4 - AI developers

AI developers build applications using APIs.

Ex -
- chatbots
- AI assistant
- PDF chat
- customer support bots
- AI search engine etc

- Most software developers work at this layer.

Large Language Model (LLM)

It is a deep learning model trained on massive amounts of text data to understand, generate & predict human language & code.

Large

It refers to the enormous amount of data & computational resources used to train the model, including, websites, books, research papers, articles & source code

Language

can understand and generate:-
- English
- Hindi
- French
- Python
- Java
- SQL

Model

Refers to a deep neural network that learns patterns from the training data & generates text by predicting the next most likely token (word or subword) in a sequence

↳ LLMs Working

Large Language Models do not search google or any search engine when answering.

Instead they:

(1.) Convert text into embeddings (vectors)

(2.) Learn statistical patterns during training

(3.) Predict the next most probable token/word.

(4) Generate response token-by-token

This process is called "Next-token Prediction".

↳ Example

- Chat-gpt
- Gemini
- claude
- Llama
- Grok
- Qwen
- Deepseek
- Mistral Large etc

→ Embedding

It is the process of converting text, words or sentence into numerical vectors that capture their meaning, allowing AI models to understand & compare them.

↳ Example:

cat → [0.21, -0.45, 0.78, ...]
Dog → [0.25, -0.41, 0.80, ...]

- **Embeddings** = These are numerical vector representation of text.

→ Langchain :

It is an open source frame-work that simplifies building applications powered by LLMs (Large language Models) by providing a common interface to interact with different AI models & tools.

↳ Why Langchain :

Different LLM providers (like openAI, Gemini, claude & groq) have different APIs, SDK, different syntax, different response format.

LangChain standarizes these interactions, allowing developers to switch between models with minimal code changes.

↳ Advantages

- common interface for multiple LLM providers
- Easy integration with AI models & external tools.
- Reusable & modular code
- Easy model switching with minimal code change
- supports prompts, memory, chains, agents & RAG Applications.
- speeds up AI application development

→ Six core components of LangChain

↳ **(1.) Models** : These are components that connect our application to LLMs & embeddings models.

- **Types:**

* **chat Models**: These generate text
  ex- chatgPT, gemini etc.

* **Embedding Models**: convert text into numerical vectors for semantic search & RAG.

* **Multimodal Models**: Process multiple data types such as text, images, audio, videos & documents.

↳ **(2.) Prompts** : These are instructions or inputs given to an LLM to generate the desired output.

- **Types:**

* **simple Prompt** : Direct Instruction
  ex- Explain Machine Learning

* **system + User Prompt**: System sets the AI's role; user asks the question.

* **Prompt Template**: Reusable prompt with placeholders
  ex- Explain [-+] with pros, cons inshort.

* **structured output**: Forces responces into formats like- JSON, XML or Tables.

(3.) Chains

It combine multiple AI tasks into a sequential workflow, where the output of one step becomes the input of the next.

• Example

Input → summarize → Translate → output

(4.) Memory

It enables an AI application to remember previous interactions & maintain conversation context.

• Example

User: My name is Aniket
Later: What is my Name?
AI : Your Name is Aniket

(5.) Indexes

It connects external data sources to LLMs, enabling them to retrieve & use information that is not part of their training data.

Used in: RAG (Retrieval - Augmented Generation)

• Sources

* PDFs
* SQL Databases
* CSV Files
* Word Documents
* Websites etc.

(6.) Agents :

These are AI systems that can autonomously decide which tools or actions to use to complete task.

• Agents

(1.) Calls the weather API

(2.) Calls the currency API

(3.) Combines both results into one responce

• Example

Question : What's the weather & convert 15 INR ?

→ steps to Install Langchain :

↳ **step : 1** - Install Python

later check latest version :- `Python --Version`

↳ **step : 2** - Install uv

uv is a fast Python package & virtual enviroment manager:

`pip install uv`

then verify installation

`uv --version`

↳ **step : 3** - Deactivate conda (if active)

`conda deactivate`

↳ **step : 4** - create a virtual Enviroment

`uv venv`

↳ **step : 5** - Activate Virtual Enviroment

`.venv\scripts\activate`

↳ **step : 6** - Install Langchain

`uv pip install langchain`

then verify installation :

```
import langchain
print(langchain.__version__)
```

API Key

It is an unique secret code that authenticates our application & allows it to securely access AI model & servies provided by cloud platforms.

Why API Keys Needed

Most modern LLMs are accessed through cloud API's because running them locally requires:

- Hundreds of GB's of storage
- High-end GPU's
- Large RAM
- significant computing power

API Keys allow developers to use these hosted models without needing powerful hardware.

Advantages

- secure access to AI services
- Easy integration with applications
- No need for expensive local hardware
- Access to the latest AI models
- Faster & scalable development

Example: in .env File

```
OPEN_AI_KEY = secret Apikey
MISTRAL_API_KEY = "  "  "  etc.
```

Universal steps to create any API Key

(1.) Visit the provider's developer platform
ex- · OpenAI
· Google AI studio
· Groq
· Mistral

(2) Sign up or log in

(3) Navigate to the API Keys or Developer section

(4) Click "create API Key".

(5) copy the generated API Key

(6) Save the API Key in a `.env` file inside our project directory.

(7) Use the API Key in our application to authenticate requests to the provider's API.

→

→ Gen AI chatbot

Now we make a chatbot using GEN AI in several steps.

↳ (1.) Creating requirement.txt & Install Dependencies

- **requirement.txt** = It is a text file that contain dependencies required by a project we can load in one shot using this.

syntax = `uv pip install -r requirement.txt`

Typical requirement for this project:

make this file inside generative AI project file:

```
langchain
langchain-community
langchain-core
langchain-openai
langchain-google-genai
langchain-groq
langchain-mistralai
langchain-huggingface
langraph
python-dotenv
streamlit
```

- **virutual enviroment** :- `python -m venv .venv`

activate it

`.venv\Scripts\activate`

then: `uv pip install -r requirements.txt`

GEN-AI CHAT BOT PROJECT

↳ overview

- LLM & API - work with cloud/local models.
- Embeddings - convert text into vectors.
- chatbot - Build conversational AI with memory.
- streamlit - create chatbot UI.
- Prompting - control model behaviour.
- chatbot. Extract structured information from text.

• Flow -

LLM → Embeddings → chatbot → UI → prompting
↓
structured AI

→ (1) Project structure

Generative_ai
- .venv
- .env
- requirement.txt
- test.py
- local_model.py
- chat_models
  - Chat.py
  - ui_chatbot.py
- embedding_models
  - embedding.py
  - huggingface_embedding.py
- cinesez
  - core.py
  - uicore.py

(2.) API Keys + .env

↳ we install :- `pip install python-dotenv`

↳ then in .env file

```
OPENAI_API_KEY = [---]
GOOGLE_API_KEY = [---]
GROQ_API_KEY = [---]
MISTRAL_API_KEY = [---]
```

↳ then in test.py file

```
from dotenv import load_dotenv
load_dotenv()
```

& check:

```
import os
print (os.getenv ("OPENAI_API_KEY") is not None)
```

(3) Chat Models

↳ Basic idea: Python → Prompt → Chat Model → Response

↳ Example: Using OpenAI model

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
load_dotenv()
model = chatopenAI (model = "enter model name")
response = model.invoke ("Explain ML")
print (response.content)
```

↳ model.invoke(..) :- sends input to the model

↳ response.content :- gets the generated text

→ (4.) Model Parameters

Syntax:-

```
model = ChatOpenAI (model = "enter model",
  temperature = 0.2, max_token = 200)
```

↳ Temperature: It controls randomness / creativity of the model's output.

- Lower Temperature = More predictable / consistent
- Higher Temperature = More creativity / variation

*** Lower for:**

- coding
- classification
- Extraction
- structured task

*** High for:**

- creative writing
- Brainstorming
- stories

↳ Max_tokens: sets the maximum amount of text the model can generate in its response.

```
ex - max_tokens = 200
```

200 maximum token output

→ (5.) Switching Providers

↳ The main pattern stays similiar:

```
response = model.invoke("Hello")
```

↳ Example: Gemini, Groq etc.

- Gemini

```
from langchain-google-genai import
ChatGoogleGenerativeAI
model = ChatGoogleGenerative(model="enter model")
```

- Groq

```
from langchain-groq import ChatGroq
model = ChatGroq(model="enter model")
```

* Remember openAI all models are paid we use free models.

↳ Main Application

Provider Changes → Model intialization charge → Application Logic → can remain Similiar

(6) Hugging Face

It provides access to many open-source Models.

↳ Install : `uv pip install langchain-huggingface`

↳ Hosted model

```
from langchain_huggingface import
HuggingFaceEndpoint

llm = HuggingFaceEndpoint(repo_id = "enter model",
      task = "text-generation")

response = llm.invoke("Explain DL")
print(response)
```

↳ Flow:

Python → Hugging Face endpoint → Hosted Model → Response

(7.) ChatHugging Face

It converts a Hugging Face LLM into a chat-model interface, making it easier to use with chat messages & Langchain's standard invoke() method

Note:- HuggingFaceEndpoint: connects to hosted interface

ChatHuggingFace: gives chat-model interface

(8) How to run "Models Locally"

↳ **first install:**

```
uv pip install transformers torch langchain-huggingface
```

↳ **Local model:**

```
from langchain_huggingface import HuggingFacePipeline
model = HuggingFacePipeline.from_model_id(
    model_id = "MODEL-ID", task = "text-generation")
response = model.invoke ("Explain AI")
print (response)
```

↳ **Flow**

Model files → Local storage → RAM/VRAM → CPU/GPU → Interface

↳ **First run:**

Model not found → Download → cache Locally → Load → Run

↳ **Some Practicle Limitations**

- **Large model needs more -**
  - VRAM
  - RAM
  - storage
  - Compute

→ (9) Embedding Model

Converts text into numerical vector that capture its meaning, mainly used for semantic search & RAG

Need is to compare text by meaning/similarity instead of just matching exact words.

↳ Most important difference

- Chat Model: Text → Vector
- Embedding vector: text → text

```
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
load_dotenv()
embeddings = OpenAIEmbeddings(
        model='text-embedding-3-large',
        dimension=64)
texts = [ any thing - - - - - - - ]
vector = embedding.embed_documents(texts)
print(vector)
```

→ (10) Local Embeddings

↳ Install:

```
uv pip install sentence-transformer
```

↳ code:

```
from langchain_huggingface import
HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
  model_name="sentence-transformers/
            all-MiniLM-L6-v2")
```

↳ Local Emedding are those model that runs on our computer instead of using a cloud API.

→ (11) embed_query()

Used for a user/search query.

```
query = "what is ML"
vector = embeddings.embed_query(query)
print(vector)
Print(len(vector))
```

↳ Flow:

User Query → embed_query() → Query Vector

→ (11.) Basic AI chatbot

```
from dotenv import load_dotenv
from langchain_mistralai import
ChatMistralAI
load_dotenv()
model = ChatMistralAI (
    model="mistral-small-2506",
    temperature=0.9)
while True:
    prompt = input("You:")
    if prompt == "0"
        break
    response = model.invoke(prompt)
    print("AI:", responce.content)
```

Goal:- create a continuous chatbot that accepts user input & generates AI responses.

- input() → takes user input
- invoke() → sends input to model
- response.content → gets AI text
- break → exits loop

↳ Flow

User Input → model.invoke() → Mistral AI → Response → response.content

→ (12.) Problem in Basic AI chatbot : No Memory

↳ If we do: `model.invoke(prompt)`

Here every request is treated independently.

Hean previous conversation wasn't sent with the new request . In short NO conversation is saved that's the reason Model / bot can't store previous into & used in another prompt / conversation.

↳ **Solution**:- store conversation history & send it with every request.

→ (13.) Basic conversation Memory

use this :

```
messages = [ ]
while True :
    pr      == "0":
    break
messages.append(prompt)
response = model.invoke(messages)
messages.append(response.content)
print("AI", response.content)
```

↳ **Flow:**

User Message → messages [ ] → Mistral AI → AI response → messages [ ]

↳ Problem with this Approach

Plain strings don't clearly tell the model:

- who said this?
- user?
- AI?
- system

For proper conversation, use message objects.

→ Langchain Messages

```
from langchain_core.message import (
    SystemMessage, HumanMessage,
    AIMessage)
```

↳ Use of Messages

For we maintain conversation history, the model needs to know who said what

Langchain_core.messages

SystemMessage	HumanMessage	AIMessage
Defines how the AI should behave during conversion	Represents the input / message given by user.	Represents the response generated by the AI model

↳ Example

```
messages = [SystemMessage(content="You are a
            helpful AI teacher."),
            HumanMessage(content="What is
             python?)"
            AIMessage(content="Python is a
            programming language?")
                ]
```

↳ conversation structure -

system → Human → AI → Human → AI

→ (15.) Streamlit chatbot

↳ first connect python application to Mistral AI API

↳ make a UI python using AI.

three emotion:

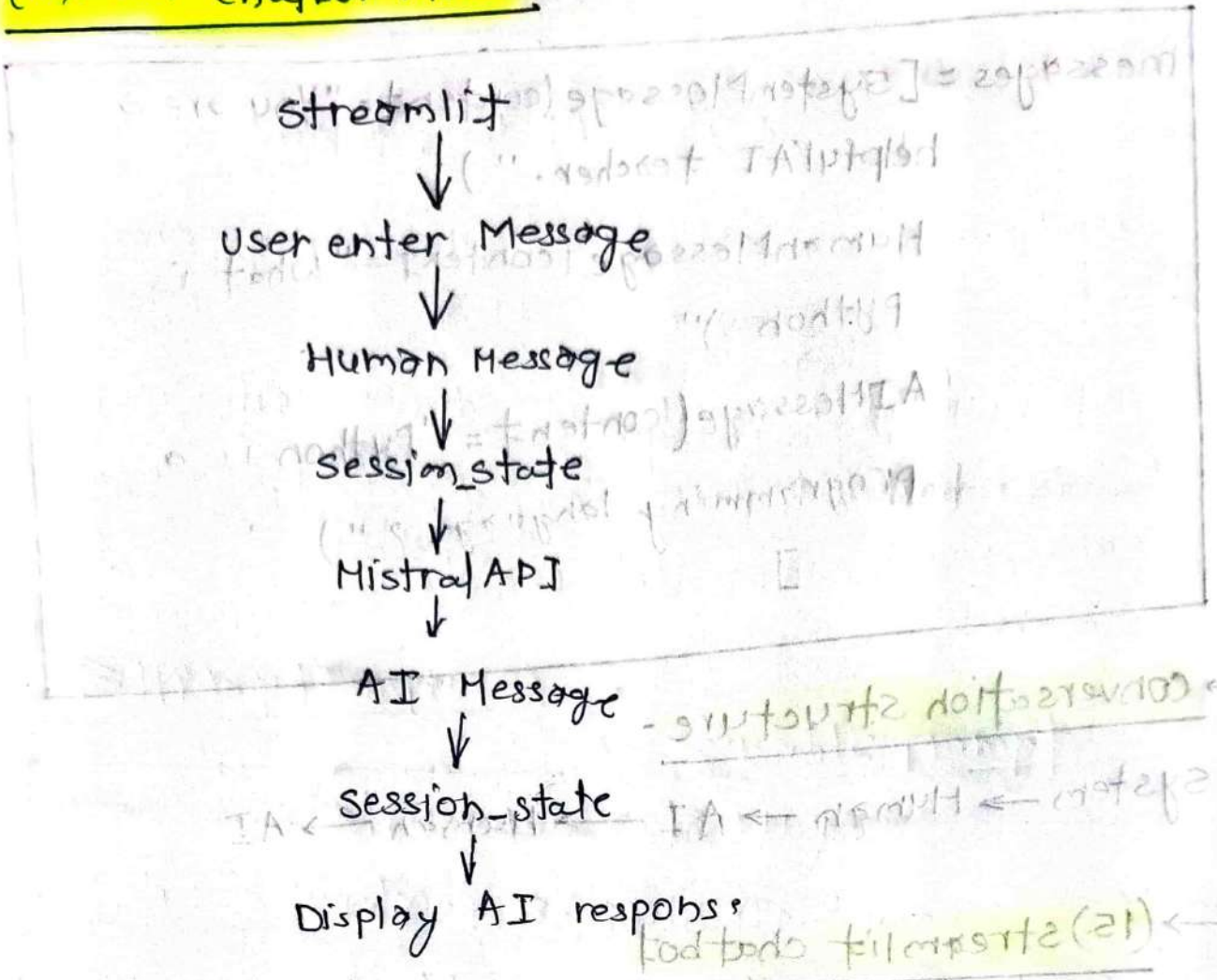
- Angry
- sad
- happy

↳ Flow

User prompt → Mistral API → AI response

The basic chatbot can take input & send it to the Mistral model

→ (16) Final chatbot flow



PROJECT-2
CINESAGE AI - MOVIE DATA EXTRACTOR

Overview

- **Goal**: Extract structured movie information from unstructured text.
- **Input**: Movie synopsis, review or description.
- **Prompt Templates**: Use reusable dynamic prompt with ChatPromptTemplate.
- **LLM**: Processes the text & extracts relevant information.
- **Pydantic**: Defines and validate the required data schema.
- **output Parser**: converts LLM output into structured data.
- **output**: clean, machine-readable JSON.
- **streamlit**: Provides a simple web interface for users.
- **Use case**: Structured data can be stored in databases or used by APIs.
- **Tech stack**: Python, Langchain, LLM API, Pydantic, streamlit

→ (1.) Prompt Templates

↳ Problem with manual Prompting

- Manually inserting large text into prompts is repetitive & difficult to scale.
- Instruction can become inconsistent.
- Not suitable for processing hundred or thousands of documents.

↳→ PromptTemplate:

It is reusable prompt structure containing:

- static instructions → remain the same
- Dynamic variables → replaced with actual data at runtime.

Ex variable : {paragraph}

↳ ChatPromptTemplate

Langchain component used to create reusable chat prompts.

Common roles:

- System → model behaviour/instruction.
- Human → user's actual input

↳ Basic Flow

Template + Runtime Data → Final Prompt → LLM

↳ Example:

```
from langchain_core.prompts import
ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages
        ([
            ("system", "You are a movie information
extractor."),
            ("human", "Extract information
from : \n {paragraph}")
                ])
result = prompt.invoke({
        "paragraph": "Interstellar was
directed by Aniket sharma"
        })
print(result)
```

→ (2.) Structured output

A feature that forces a model to return data in a strict, predefined format.

commonly JSON.

↳ Problem with Normal LLM output

- LLMs return unstructured natural language.
- Difficult to use directly with databases, API, & software.
- output format is not guaranteed or consistent.

eg- Avengers endgame is scifi action movie directed by aniket.

↳ **Advantages**

- Machine-readable
- consistent
- Easier to validate
- Easier to store
- Easier to process programmatically

↳ **Example:**

```
movie = {
    "title": "Endgame"
    "release_year" :"2021"
    "director" :"Aniket"
print (movie["title"])
```

→ **(3.) Pydantic**

It is a python library used for defining & validating structured data.

A Pydantic model defines

- Field names
- Data types
- Required loptional fields
- Expected structure

↳ **Why Pydantic is useful with LLMs**

LLM can return sometimes:

- Wrong data types
- Missing fields
- Incorrect formatting
- Unexpected information

• Pydantic provides a validation layer.

LLM output	→ Pydantic
Validation	→ structured Data

↳ Example schema:

```
from pydantic import BaseModel
from typing import List, optional

class MovieSchema(BaseModel): title :str
    release_year : Optional [int]=None
    genre : List [str]
    director : optional [str] =None
    cast : List [str]
    rating : optional [float] = None
    summary : str
parser= PydanticOutputParser (Pydantic_object = Movie)
```

→ (4.) PydanticOutputParser

PydanticOutputParser connects LangChain with a Pydantic schema.

↳ Main Purposes

(1.) Defines the expected output format.
(2) Generates formatting instructions.
(3.) Parses the LLM response.
(4) Validates the response.
(5) Returns structured Python data

Example

```
from langchain_core.output_parsers import
PydanticOutputParser
parser = PydanticOutputParser(pydantic_object=
        MovieSchema)
```

Format Instructions

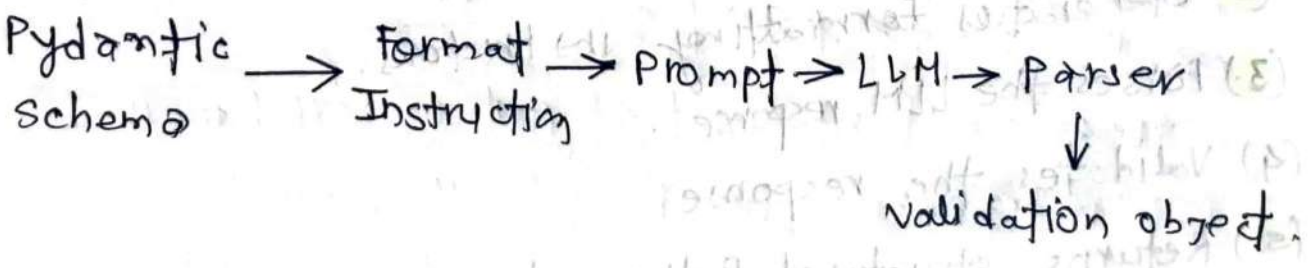
The parser can automatically generate instructions for the LLM.

```
format_instructions = parser.get_format_
        instructions()
print(format_instructions)
```

These instructions tell the model about:

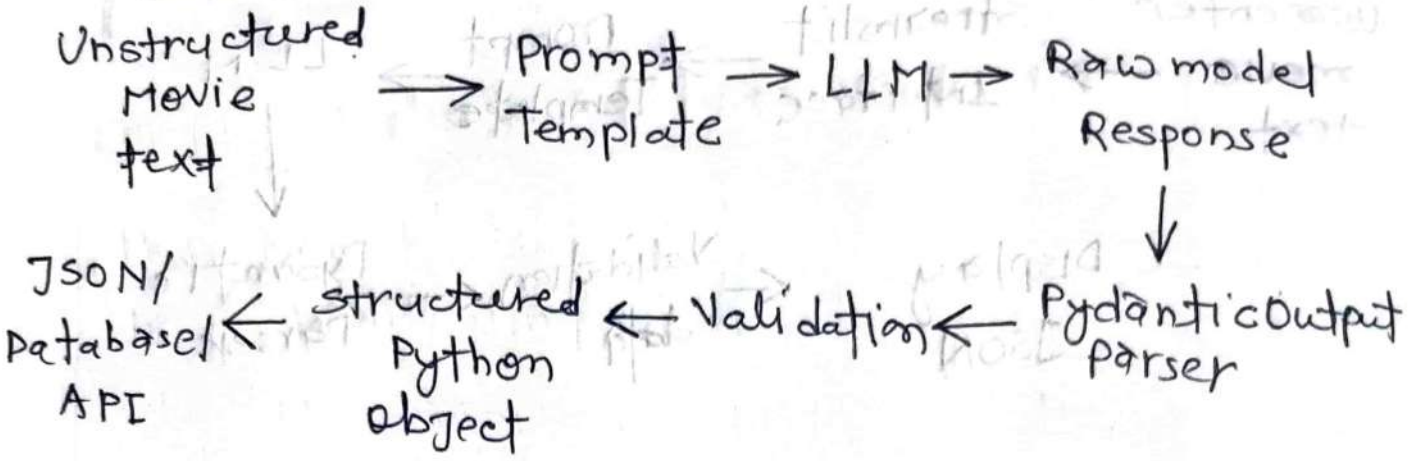
- Required fields
- Expected structure
- Data types
- Formatting requirements.

Flow



→ (5.) Complete Structured Extraction Pipeline

↳ Architecture



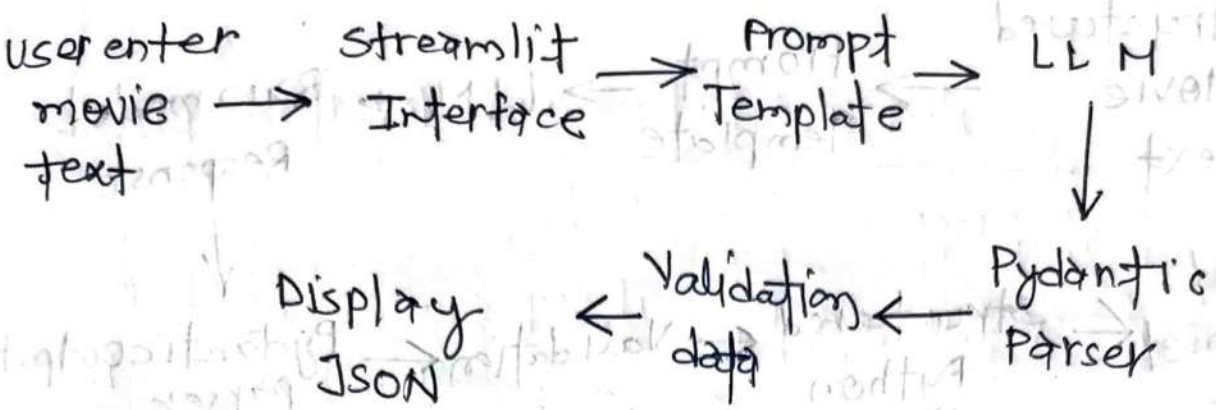
↳ Final Combined Example

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.outputs_parsers import
PydanticOutputParser

parser = PydanticOutputParser(pydantic_object=
        MovieSchema)
prompt = ChatPromptTemplate.from_messages([
    ("system", "Extract movie information.\n
    {format_instructions}"),
    ("human", "Movie text:\n{paragraph}")])
final_prompt = prompt.invoke({
  "paragraph": "Insteller is a 2014 film directed
                by Aniket",
  "format_instruction": parser.get_format_
      instructions()})
```

→ (6.) Streamlit Integration

↳ Application Flow



↳ Make a suitable UI using AI give him the core.py or core code to understand more about

→ **Most Important for streamlit**

↳ **Resource caching** : Prevents streamlit from reloading/ recreating expensive resource (like LLM model) on every re-run

```
@st.cache_resource
def load_model():
    return YourModel()

model = load_model()
```

Flow

Imports → st.cache_resource
↓
Streamlit UI ← model = loadmodel()
↓
user interface (UI)

Put **@st.cache_resource** around the function that creates / loads your model, near the top of our streamlit file, after imports.

PART-2

RAG (RETRIVAL-ARGUMENTED GENERATION) & DOCUMENTS LOADERS, TEXT SPLITTERS, VECTOR DB

→ RAG (Retrival Argumented Generation)

It is a technique that helps an LLM answer questions using information from external sources.
such as PDFs, documents, databases or company files.

- Instead of depending only on the knowledge it learned during training, the LLM first searches for relevant information, retrives the useful parts & then uses that information to generate the answer.

→ **Example:**

If we have 500 page PDF about ML.

If we ask: "What is Gradient Descent"

RAG: Instead of sending all 500 pages to the LLM

Question → Search PDF → Find relevant paragraph

↓

Send paragraph + Question to LLM

←

↳ Advantages of RAG

- **Up-to-date knowledge**: can use recently added information
- **Private data**: can answer using our own documents
- **More accurate**: Answers are grounded in retrieved context.
- **Less hallucination**: Reduces the chance of making up information.
- **Cost efficient**: Sends only relevant chunks instead of entire documents.
- **Easy to update**: Update the knowledge base without retraining the LLM.

PROJECT : 1
COURSE-MATE AI

Overview

- AI powered study assistant for chatting with PDF's, notes, textbooks & research papers.
- Uses RAG to retrieve relevant information before generating answers.
- **Document loading** : PyPDFLoader & TextLoader
- **Text chunking** : split large documents into smaller chunks.
- **Embeddings** : convert chunks into numerical vectors.
- **Vector databases** : store embeddings in chromaDB.
- **Retrieval** : Uses MMR & MultiQuery Retriever for better results.
- **LLM** : Mistral AI generate answers from retrieved context.
- **UI** : streamlit for document upload & chat.
- **Tech stack** : Python, Langchain, openAI Embeddings, Mistral AI, chromaDB, streamlit, uv.
- **Benefit** : Quickly answers questions from large study materials without manual searching.

→ RAG Development Pipeline

The RAG system can be understand as 10 step pipline.

↳ **step: 1** - Upload Document, user uploads:
- PDF
- TXT
- Notes
- Research paper etc.

↳ **step: 2** - Document Loading

The files converted into Langchain Document objects.

↳ **step: 3** - Text splitting (chunking)

Large documents are divided into smaller chunkes

Ex- 500 page book → small text chunk → chunk 1, chunk 2, ..., chunk n

↳ **Step: 4** - Embeddings

Each chunks is converted to a numerical vector using embedding model.

ex - Text (A,B,C,D) → Embedding Model → [0.21, -0.43, 0.87, 0.71]

↳ **step: 5** - Vector Database

embedding stored in a vector database.

Ex - chromaDB, It database generally stores:

Embedding + original Text + Metadata

→ **Step : 6** - User Question

When user asks: "What is Perceptron"

→ **Step : 7** - Query Embedding

The Question is also converted into Embeddings.

Ex

"What is Perceptron" → Embedding Model → Query vector

→ **Step : 8** - similarity search

- The Query vector is compared with stored vectors.
- The system searches for chunks that are semantically similar.
- One common similarity measure is: cosine similarity

→ **Step : 9** - Retriever

It returns the Top K relevant chunks

Ex -

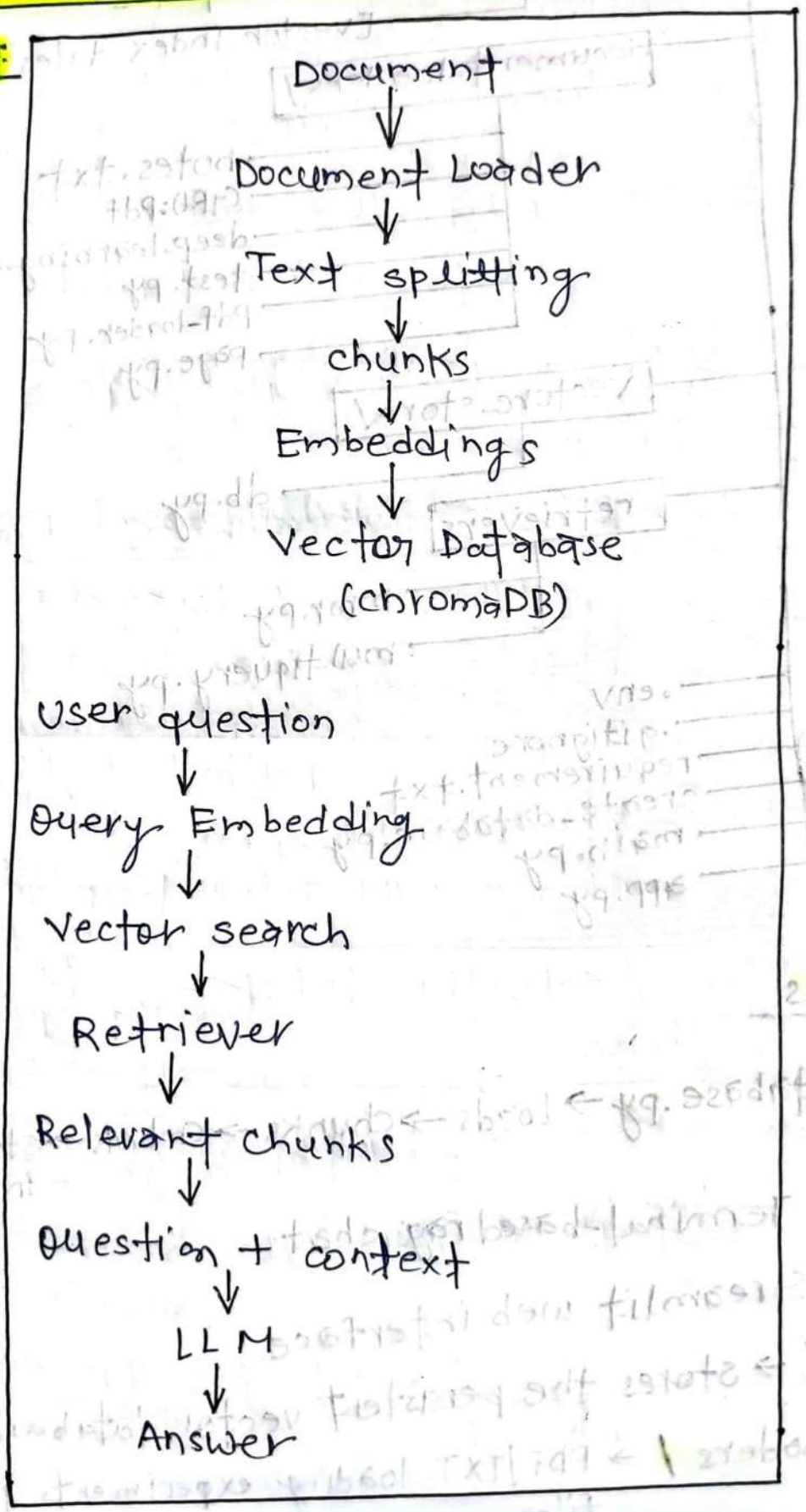
Question → vector search → Top 3 relevant chunks

→ **Step :10-** LLM Generation

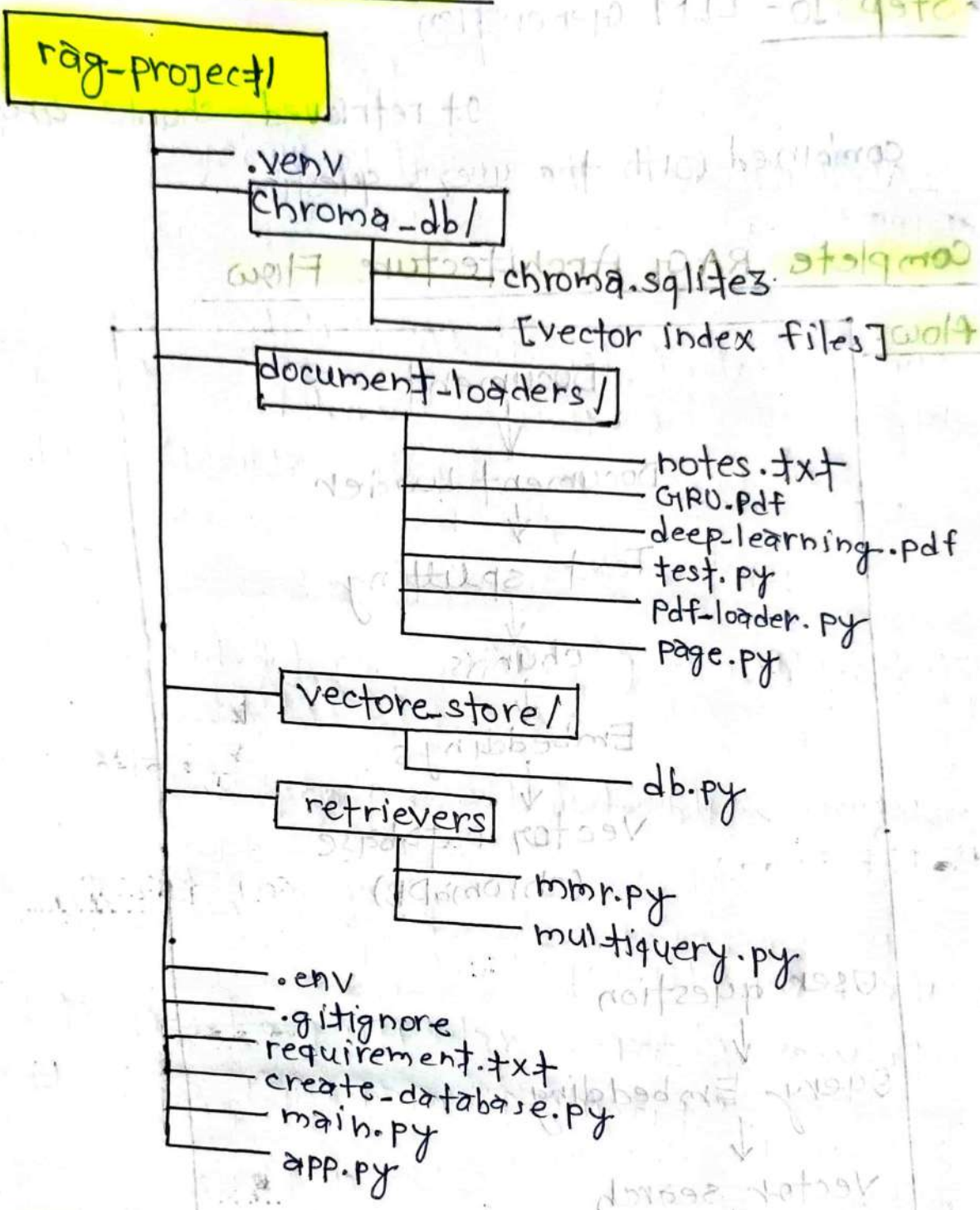
Of retrieved chunks are combined with the user's question.

→ **complete RAG Architecture Flow**

flow:



→ Complete File structure



↳ Main Files

- create_database.py → Loads → chunks → embeds → stores documents in chroma DB
- main.py → Terminal-based RAG chat.
- app.py → Streamlit web interface
- chroma_db/ → stores the persistent vector databases
- document_loaders / → PDF/TXT loading experiments & study files.

- vector_store/db.py → chromaDB/vector-store experiments.
- retrievers/mmr.py → MMR Retrieval.
- retrievers/multiquery.py → MultiQuery retrieval.
- .env → API Keys
- requirements.txt → Required Python libraries.
- .venv/ → Isolated Python environment.

→ Required Dependencies

The Project uses:

```
langchain
langchain-core
langchain-community
#LLM Providers
langchain-openai
langchain-mistralai
openai
mistralai
#Vector Database
chromadb
#Embedding Models
sentence-transformers
#Document Loader dependencies
PyPdf
unstructured
beautifulsoup4
lxml
#Text Preprocessing
tiktoken
```

```
#Environment Variables
python-dotenv
#Data handling
pandas
numpy
# API/ Application
fastapi
uvicorn
##UI
streamlit
#Utilities
tqdm
requests
```

→ Testing Mistral API

```
from dotenv import load_dotenv
from langchain_mistralai import ChatMistralAI

load_dotenv()
model = ChatMistralAI (model="mistral-small")
result = model.invoke("Hello")
print(result.content)
```

↳ **Flow -**

.env → load_dotenv() → API key available → chatMistral AI ↓

Response ← Mistral API ← model.invoke()

→ Document Loaders

It is responsible for reading data from a particular source & converting it into Langchain "Document".

Ex -

TXT → TextLoader
PDF → PyPDFLoader

The loader abstracts away much of the file-reading logic.

→ TextLoader

for .txt files:

```
from langchain_community.documents_loaders import
TextLoader
loader = TextLoader ("document_loaders/notes.txt")
docs = loader.load
print (docs)
```

Note - "load()" normally returns a list of "Document" objects

Therefore if "docs" is a list:

```
docs[0].page_content
```

it is used to access the first document's cont.

Not:

```
docs.page_content
```

→ Text File → LLM Summary

Loaded content can be passed into a prompt.

```
from langchain_core.prompts import ChatPromptTemplate
template = ChatPromptTemplate.from_messages([
  ("system", "you are an AI that summaries the text."),
  ("human", "{data}")
])
final_prompt = template.format_messages(data=docs[0].
                page_content)
result = model.invoke(final_prompt)
print(result.content)
```

↳ Pipeline

TXT File → Text Loader → Document → page_content

↓

Summary ← LLM ← Prompt Template

→ PyPDFLoader

It is a langchain document loader used to read PDF files & convert them into Langchain Document objects.

```
from langchain_community.document_loaders
import PyPDFLoader
loader = PyPDFLoader ("document_loader/CIRO.pdf")
docs = loader.load()
len(docs) # if pdf contains 15 pages it give
            output 15
```

→ Metadata

It is extra information about the data like- In langchain PDF Document, metadata can include

- source → PDF file path
- page → page number
- total_pages → total pages

↳ **PDF Metadata**

Each page's Document can contain metadata.

ex-

```
print(docs[0].metadata)
```

→ **PDF content**

↳ To access a particular page:

```
print(docs[0].page_content)
```

↳ For another page:

```
print(docs[5].page_content)
```

↳ If docs is a list then

```
docs[5]. #selects a Document
docs[5].page_content # get it text
```

→ **The context Window Problem**

Let assume:

a textbook contains
534 pages
20MB

sending the full/entire notebook to LLM for every question is bad architecture.

Problems

(1.) **context Limit**: The Model May not be able to process the entire document within its context window.

(2.) **cost** : sending thousands or millions of tokens repeatedly increases API costs

(3.) **Latency**: Large prompts require more processing

(4) **Irrelevant Information**: The question might concern only one page, while the prompt contains hundreds of irrelvant pages

(5) **Rate Limits**: Huge requests can contribute to API / rate-limit problems

→ Why RAG solves the Problem

Instead; 534 Pages

↓

LLM ⟶ Answers

the RAG does:

534 Pages

↓

chunking

↓

Embeddings

↓

vector database → User question

Relevant 3-5 chunks ⟶ LLM

↑ ↓

Retrival Answer

↑

→ **Web based Loader**

It loads text/context from a web page (URL) & converts it into LangChain Document objects

```
from langchain_community.document_loaders
import WebBaseLoader

loader = WebBaseLoader("https://aniket.com")
docs = loader.load
```

Flow:

Web Page → WebBaseLoader → Documents

↓

Vector DB ← Embeddings ← Chunking

*doc.metadata typically contains the source URL & other page-related information.

→ Text splitter

It's a tool that divides large documents into smaller chunks so they can efficiently embedded, stored, retrieved & processed by an LLM.

↳ Why Text splitting (chunking) is Necessary

(1.) Context Window Limit : LLMs have a Maxm no. of token they can process in a request

ex -

534 Page Pdf
↓
Huge prompt
↓
context overflow/ API errors

Large requests can also contribute to 429 rate-limit errors, depending on the API & usage limits.

(2.) Semantic Dilution : suppose one huge document contains:

Deep Learning
Computer Vision
NLP
RLHF
ML

embedding the entire document can produce a representation that mixes many different concepts

Each embedding can represent a more focused concepts.

Instead we do:

chunk-1 : Deep Learning
chunk-2 : Computer Vision
chunk-3 : NLP
- - etc

(3.) Better Retrieval

If aniket ask: "What is Back-propagation"

we don't want vector database to return an entire 500 page text book.

we want:

Question → similarity search → Relevant chunks → LLM

only the most relevant passages are provided as context.

(4) speed & cost

Smaller chunks generally make the retrieval pipeline more manageable:

- less un-nessary context
- more target retrieval
- lower prompt-token usage
- faster processing
- more efficient similiarity search

↳ Chunking & its parameter

Chunk: A smaller section of a large document created by a text splitter. chunks are later converted into embeddings & stored in vector databases.

Important Parameter

- **chunk_size** - Max^m / target size of each chunk

chunk_size = 1000

Depending on the splitter it can represent characters or tokens.

• chunk_overlap : Amount of content shared between consecutive chunks

```
chunk_overlap = 200
```

It preserves context between chunks & prevents important information from being lost at boundaries.

↳ **Types of Text splitters**

(1.) **Character-based splitting**: simplest splitting text according to character

Ex-

```
from langchain_text_splitters import
CharacterTextSplitter

splitter = CharacterTextSplitter(
        seperator = " ",
        chunk_size = 10,
)       chunk_overlap = 1

chunks = splitter.split_documents(docs)
```

• separator : Boundary at which text is split or means the splitter uses spaces as boundary.

Note :- character-based splitting is relatively simple & does not understand the deeper semantic structure of the document.

(2) Token-Based splitting

Here LLM's process text as tokens, not simply character. It divides documents according to token count.

Ex-

```
from langchain_text_splitters import
TokenTextSplitter

splitter=TokenTextSplitter(
        chunk_size=100,
        chunk_overlap=10 )
chunks = splitter.split_documents(docs)
```

* Token & tiktoken

- **Token**: It is a piece of text processed by a language model.

the token can be:

- a complete word
- part of a word
- punctuation
- whitespace-related text

flow

Text → Tokenizer → Token 1, Token 2, Token 3

- **tiktoken**: It is OpenAI tokenizer on Byte Pair Encoding (BPE)

It is useful when working with openAI models because token-based chunk size can be aligned more closely with model token limit

* Token count is therefore more directly related to LLM context window

(3.) Recursive Character Text Splitter

It is one of the most commonly used general-purpose splitters in RAG systems.

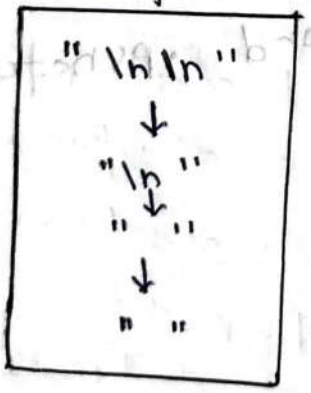
Ex-

```
from langchain_text_splitters import
RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size= 1000,
    chunk_overlap =200 )
chunks=splitter.split_document(docs)
```

It doesn't immediately cut text at arbitary positions.

It tries progressively smaller separators.

Default hierarchy:



↳ Working of Recursive Splitting

suppose:

```
chunk_size = 1000
```

- **Level 1 - Paragraph**

`\n\n` → It attempts to keep complete paragraphs together.

If a resulting piece is still too large:

- **Level 2 - Line**

`\n` → if it is still too large

Note:- `\n` → New line / Line Break

`\n\n` → Two new lines / blank line (paragraph break)

- **Level 3 - Word**

`" "` → If even that is too large.

- **Level 4 - character**

`""`

Therefore:

Paragraph
↓
Line
↓
Word
↓
character

This approach attempts to preserve the natural structure of the text as much as possible.

(4.) Semantic Splitting

It is a more advanced approach. A text-splitting method that divides a document based on meaning & topic changes, rather than fixed character or token sizes.

Ex-

```
from langchain_experimental.text_splitter
import SemanticChunker

from langchain_openai import OpenAIEmbeddings.

embeddings = OpenAIEmbeddings()
splitter = SemanticChunker(embeddings,
    breakpoint_threshold_type="percentile")
chunks = splitter.split_documents(docs)
```

↳ Working of Semantic splitting

Let assume:

sentence 1	→ Deep Learning
sentence 2	→ Neural Network
sentence 3	→ Backpropagation
sentence 4	→ NLP
sentence 5	→ Transformers

- **Step: 1** - It Breaks the document into sentences.

  S1, S2, S3, S4, S5

• **Step: 2-** creates Embeddings

S1 → E1 , S2 → E2 , S3 → E3 , S4 → E4
S5 → E5

• **step: 3-** compares neighboring embeddings.

similarity (E1, E2)
similarity (E2, E3)
similarity (E3, E4)
Similarity (E4, E5)

• **step: 4-** Uses a similarity threshold.

For example-

Threshold = 0.5

If - similarity = 0.8
0f sentences are likely related
If - similarity = 0.4
there may be a topic transition.

↳ Disadvantages of Semantic Splitting

• **Computationally expensive** - requires embeddings for sentences
• **Slower** - performs similarity calculations.
• **Higher cost** - embeddings generation adds API/compu cost
• **More complex** than simple character-based splitti

→ CourseMate AI - Implementation

```
from dotenv import load_dotenv

from langchain_community.document_loaders
import PyPDFLoader
from langchain_text_splitters import
RecursiveCharacterTextSplitter
load_dotenv() #Load enviroment variable
#Load PDF
loader = PyPDFLoader("document_loader/
        deep_learning.pdf")
docs = loader.load()
#split document into chunks
splitter = RecursiveCharacterTextSplitter(
        chunk_size =1000,
        chunk_overlap = 200)
chunks = splitter.split_documents(docs)
print(f"Pages : {len(docs)}")
print(f"chunks : {len(chunks)}")
```

Flow :

PDF → Documents → Text splitter → chunks

→ Vector stores & Vector Databases

- **Vector store**: A system that stores embeddings & provides similarity search to retrieve relevant data.

- **Vector Databases**: It is a specialized database designed to store, index & efficiently search high-dimensional vector embeddings.

Ex-

100000 chunks
↓
100000 embeddings
↓
User query → query Embedding
↓
compare with stored vectors

↳ Traditional DB Vs Vector DB

Traditional Database	Vector Database
(1) Structured Data	(1) Vector embeddings
(2) Exact / keyword search	(2) Semantic search
(3) B-Tree / Hash indexes	(3) HNSW / IVF / PQ
(4) SQL queries	(4) similarity search
(5) Transactions & structured records	(5) AI / RAG / Recommendation

↳ Approximate Nearest Neighbour (ANN)

It is a technique used by vector databases to find vectors that are approximately closest to a query without comparing against every stored vector.

Normal Technique

Query
↓
Compare with all vectors
↓
Result

Approximate Nearest Neighbour

Query
↓
Vector Index
↓
Relevant search Area
↓
Nearest Vectors

It uses an index to narrow down the search area.

This makes large-scale vector search much faster.

↳ Important Vector indexing Algorithms

(1) HNSW - Hierarchical Navigable Small World

- Uses a Multilayer graph.
- Upper layer provide fast Navigation,
- Lower layer provide more detailed searching.
- Commonly used for fast approximate nearest-neighbour search

Top Layer → Fast routing
↓
Middle Layer → Narrow search
↓
Bottom Layer → Detailed search

* Most Used Algorithm

(2) IVF - Inverted File Index

- Groups vectors into clusters.
- Uses clusters centroids.
- Searches relevant clusters instead of the entire database.

Ex-

100000 vectors
↓
5 - clusters
↓
Query → Find Relevant cluster
↓
Search only that cluster

(3.) PQ - Product Quantization

- Comparesses vectors into smaller representation.
- Reduces memory usage.
- can speed up distance calculations.
- Useful for very large vector collections.

Flow

Large vector → split into smaller subvectors → Quantize/Compare each subvector → Stores compact codes → Fast similarity search

→ ChromaDB

It is an open-source vector database / vector store commonly used for AI applications & RAG.

↳ Advantages:

- Easy to use
- works well with Langchain
- can run locally
- supports persistent storage
- Provides similarity search

↳ Flow

"What is Epochs"
↓
Query Embedding
↓
ChromaDB
↓
Find similar vector
↓
"Epochs is Back + Forward Propagation"
↓
LLM
↓
Final Answer

chromaDB stores embeddings & retrieves the most semantically similiar chunks when a user asks a question.

↳ some parameter

- **K Parameter** : K specifies the no. of top result to retrieve

Ex -

K=2	→ 2 results
K=3	→ 5 results
K=10	→ 10 results

- **persist_directory** : saves the vector database locally for future use.

persist_directory = "chroma_db"

- saves the chromaDB persistently on disk.
- Data remains available after the application restarts.
- creates a local chroma_db/ directory.

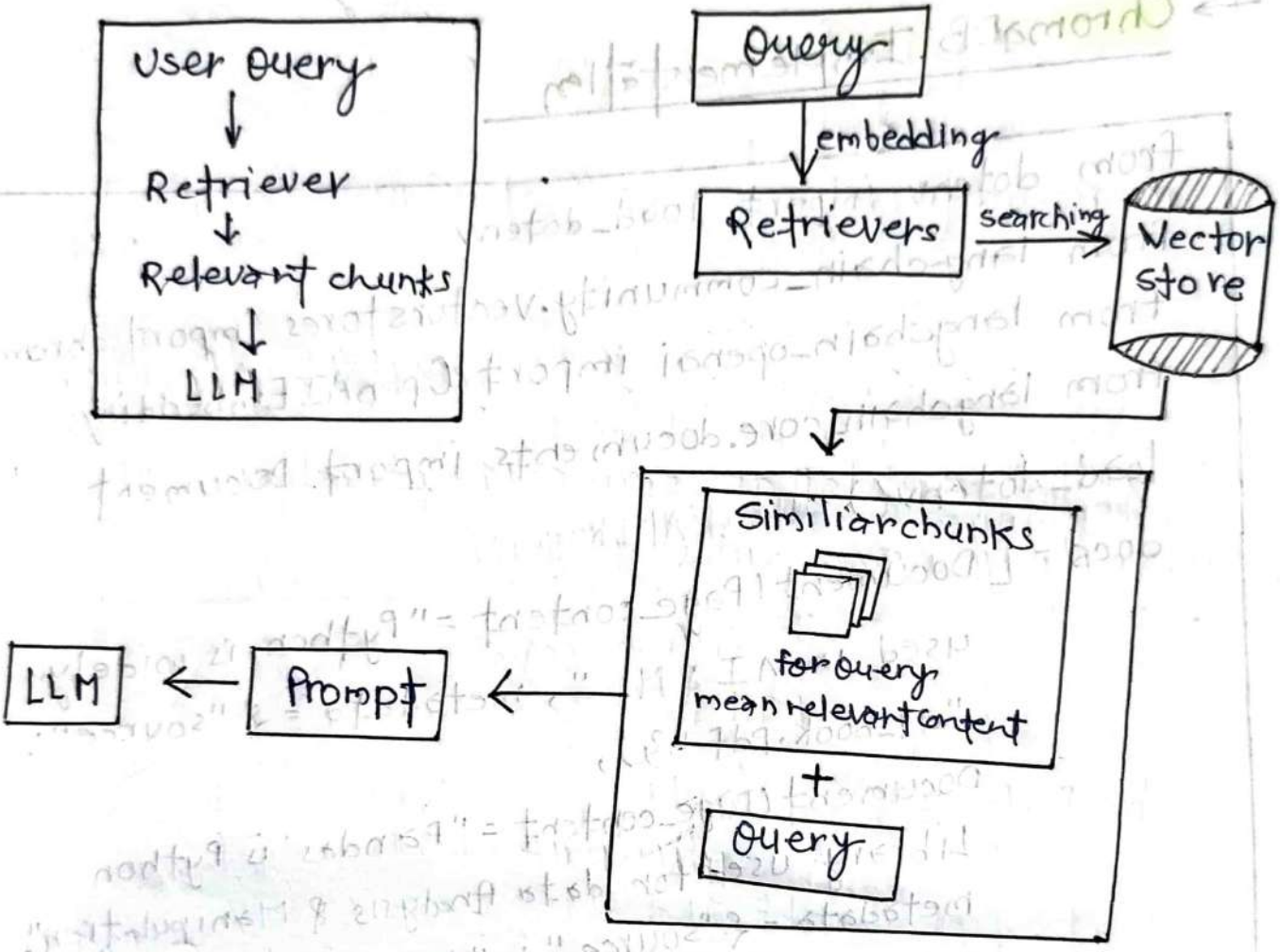
ChromaDB Implementation

```
from dotenv import load_dotenv
from langchain_community.vectorstores import chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document

load_dotenv() #Load APIKey
#creating docs
docs = [Document(page_content="Python is widely
        used in AI & ML", metadata={"source":
        "ai_book.pdf"}),
       Document(page_content="Pandas is Python
        Library used for data Analysis & Manipulation",
        metadata={"source": "data-science-notes.txt"}),
       Document(page-content="Neural Network are
        computational model used in Deep Learning",
        metadata={"source":"deep-learning-hangout.pdf"}
        )]
embedding_model = OpenAIEmbeddings()
#Embedding model
vector_store=chroma.from_documents(documents=docs,
    embedding=embedding_model, persist_directory=
    "chroma_db") #create & persist vector store
query = "What is SVM"
result = vector_store.similarity_search(query, k=2)
for doc in results:
    print(doc.page_content)
    print(doc.metadata)
```

→ Retriever

It is a component that fetches relevant documents/chunks from a data source for a user's query.



→ Vector store Vs Retriever

(1) Stores & searches embeddings	(1) Retrieves relevant documents
(2) Input → Vector/documents	(2) User query ← Input
(3) output → similiarvector/documents	(3) output → Relevant chunks
(4) Role → Data storage & search	(4) Role → Retrieval layer
(5) Ex - chromaDB	(5) Ex → MMR Retriever

↳ Types of Retrievers

(1.) Data source Retrievers

It Retrieve information directly from external sources.

Ex - Wikipedia, Arxiv, PubMed etc.

Ex -

```
from langchain_community.retrievers import
ArxivRetriever
retriever = ArxivRetriever (load_max_docs = 3)
docs = retriever.invoke ("Large Language Model")
```

Flow:

Query ⟶ ArXiv ⟶ Relevant Papers

(2) Retrieval strategy Retrievers

It controls how documents are selected from a vector store.

Main strategies:

(A) similarity search

(B) MMR (Maximum Marginal Relevance)

(C) MultiQuery

Flow:

User query ⟶ Retriever ⟶ Retrieval strategy

↓

LLM ⟵ Relevant chunks ⟵ Vector store search

(A) Similarity Search

Finds the documents whose vector are most similiar to the query vector. Most used.

• Flow

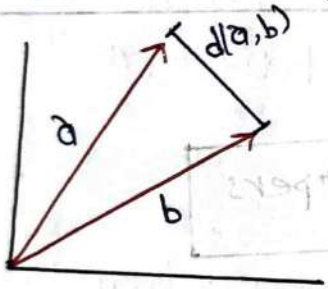
Query → Query Embedding → Vector Database

↓

Top K Documents ← Similarity Comparison

• Similarity Metrics

(i) Cosine similarity - Measure angle/dirn betⁿ vectors
Higher value = more similiar meaning



$$\cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

Most commonly used similarity.

(ii) Dot Product - Measures how strongly two vector align
Higher value = great similarity

(iii) Euclidean Distance - Measures the straight line distance betⁿ vectors.
Smaller distance = more similiar.

• syntax

```
results = vector_store.similarity_search(query, k=4)
```

• Example

query : "what is Gradient Descent?"

chunk 1 → optimization algorithm

chunk 2 → Minimizes loss Algorithm

chunk 3 → Minimizes loss function

chunk 4 → used for neural Network training

(B) MMR - Maximum Marginal Relevance

It Balances relevance & diversity.

Instead of returning only highly similar chunks,

MMR tries to return chunks that are:

- relevant to the query
- different from each other

• Flow

query → Fetch candidate chunks → MMR selection

↓

Relevant + Diverse chunks

↓

LLM

• Ex

```
retriever = vector_store.as_retriever (search_type="mmr",
            search_kwargs={"k": 4, "fetch_k": 10,
            "lambda_mult": 0.5})
```

- **Parameters**

(i) fetch_K = No of initial candidates

(ii) K → final no. of documents returned

(iii) lambda_mult → Controls relevance Vs diversity

lambda ≈ 1 → More relevance
lambda ≈ 0 → More diversity

(c) MultiQuery Retriever

It uses an LLM to generate Multiple versions of the user's question, then searches using those variations.

- **Ex-**

Original Query: "What is gradient descent"
Generated queries: "Explain gradient descent work?"
"How does gradient descent work?"
"Which optimization algorithm updates neural Network weights?"

- **Flow-**

User Query → LLM → Multiple Query variations

↓

Vector Database searches

↓

Relevant Documents ← Combine + Deduplicate Result.

• code -

```
from langchain.retrievers.multi_query import
MultiQueryRetriever

mq-retriever = MultiQueryRetriever.from_llm (
    retriever = vector_store.as_retriever () ,
    llm = llm)
docs = mq-retriever.invoke("What is gradient descent")
```

→ Final Completation

↳ Here is the "main.py" file

```
from dotenv import load_dotenv
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import
Chroma
from langchain_mistralai import ChatMistralAI
from langchain_core.prompts import ChatPromptTemplate

load_dotenv()

embedding_model = HuggingFaceEmbeddings(
    model_name = "sentence-transformer/all-MiniLM-
                  L6-v2")

vectorstore = Chroma(
        persist_directory = "chroma_db",
        embedding_function = embedding_model)

retriever = vectorstore.as_retriever (
        search_type = "mmr", search_kwargs = {
        "k":4, "fetch_k":10, "lambda_mult":0.5})

llm = ChatMistralAI (model="mistral-small-2506")

prompt = ChatPromptTemplate.from_message ([
        ("system", """You are a helpful ai assistant """),
        ("human", """context : {context}
        Question : {question} """)])
```

```
print("Rag system.created")
print("press 0 to exit")
while True:
    query = input("you:")
    if query == "0":
        break
    docs = retriever.invoke(query)
    context = "\n\n".join(
        [doc.page_content for doc in docs])
    final_prompt = prompt.invoke({
        "context": context,
        "question": query})
    response = llm.invoke(final_prompt)
    print(f"\n AI: {response.context}")
```

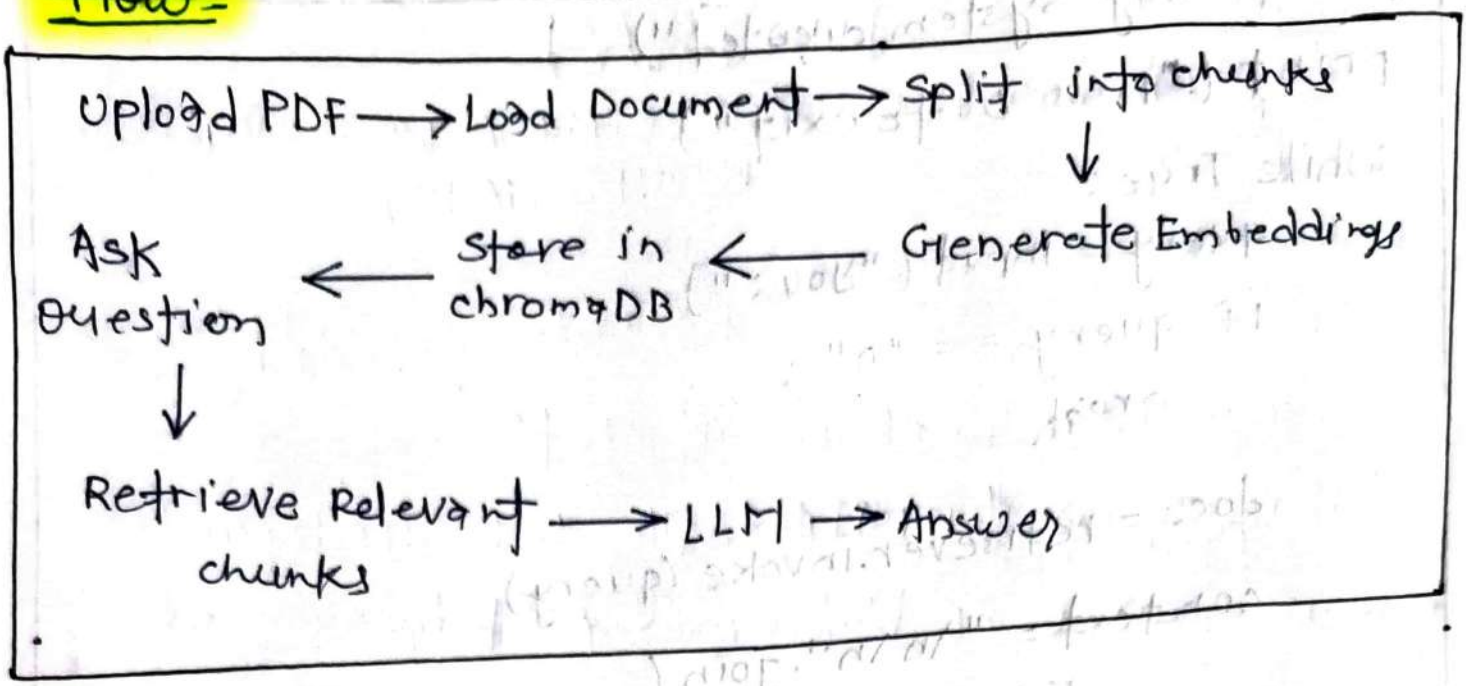
↳ **Streamlit UI**

Streamlit converts the terminal RAG Application into a web-based graphical interface.

Main features:

- **PDF uploader** - Upload study material
- **Database Builder** - Loads, chunks, embeds & stores the document
- **Chat interface** - Ask Questions about the uploaded document.
- **Processing indictor** - shows when the system is working

Flow -


Upload PDF
Load Document
Split into chunks
Generate Embeddings
Store in chromaDB
Ask Question
Retrieve Relevant chunks
LLM
Answer

PART-III

TOOLS, AI AGENTS, TOOL CALLING, API'S & LANGCHAIN

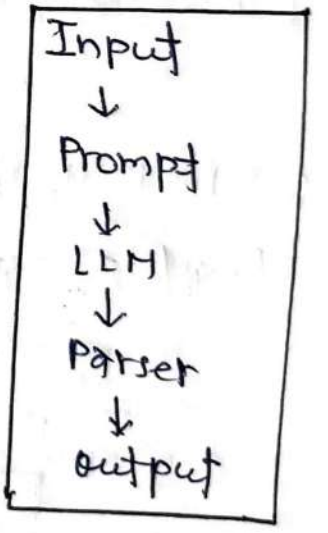
→ Runnable

It is a standard interface in Langchain that allow different components to be executed & connected in a common way.

In short- "Anything that can take input & produce an output can be treated as a Runnable".

Ex-
- Prompt Template
- LLMs
- Chat models
- Output parsers
- Retrievers
- Tools
- custom Python functions.

↳ like Each component as a LEGO Block-



↳ Why we need Runnables ?

Before Runnable langchain has many approx 50+ chains . each for different Usecase .

Problem 1 - Too many chains

developers had to remember which chain was appropriate for their use case.

If usecase doesn't match with chain then mixup chains or use existing non appropriate chain.

Problem 2 - Rigidity

Suppose we won't

Prompt → LLM → custom Python Function → Another LLM

A predefined chain might not provide exactly this workflow.

Problem 3 - Less control

Adding our own logic Bet^n different steps could become difficult.

↳ **Why Runnable is solution?**

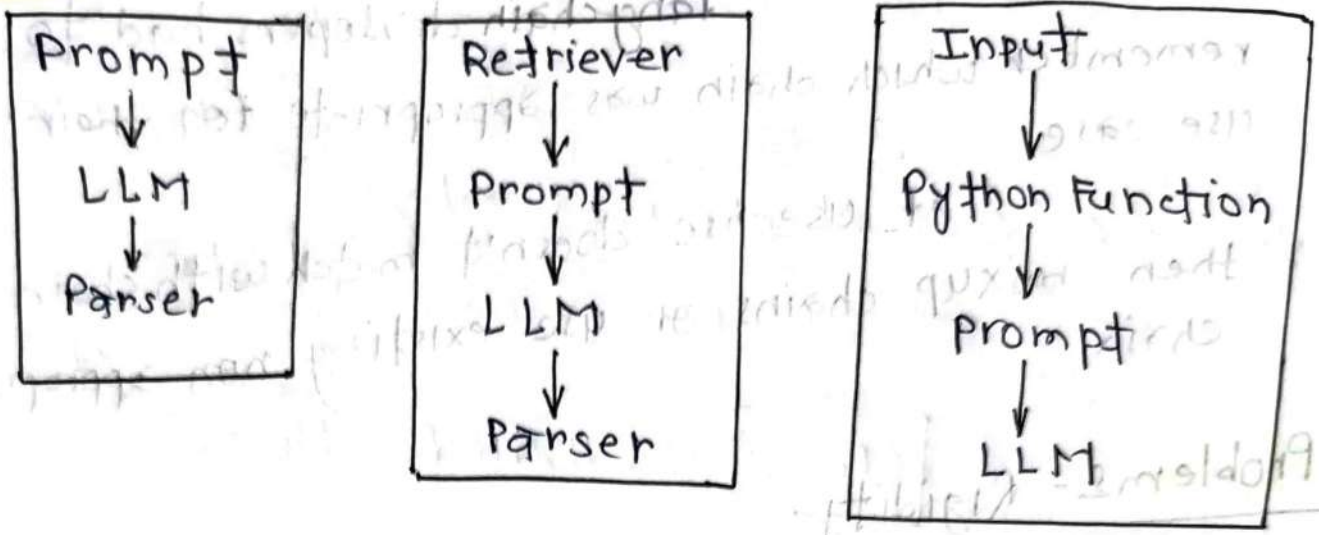
Because Instead of creating a separate class for every possible workflow, Langchain provides small reusable components.

- Each component can work as a Runnable.

Prompt → Runnable
LLM → Runnable
Parser → Runnable
Parser → Runnable
Retriever → Runnable
Function → Runnable

- Runnables can combined to create custom workflow.

Ex-



Simple Term :- Instead of choosing a predefined chain, we build our own workflow by connecting Runnables.

→ **LCEL (Langchain Expression Language)**

It is the syntax used to compose Runnables together.

- It connect & compose Runnables into workflows.
- It uses pipe operator "|" to pass output from one Runnable to the next.

Ex-

```
chain = prompt | model | parser
```

Flow:

Prompt → Model → Parser

→ Pipe operator (|)

It connects Runnables in LCEL.

- output of one Runnable automatically becomes the input of the next.

Ex-

```
chain = prompt | Model | parser
```

Flow-

Input → Prompt → Model → Parser → Output

→ Runnable sequence

It executes runnables one after another in a fixed order.

It is the most basic LCEL workflow.

Ex-

```
chain = prompt | model | parser
```

Flow

Input → Prompt → Model → Parser → Output

code-

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_mistralai import ChatMistralAI
prompt = ChatPromptTemplate.from_template("Explain {topic}
        in simple words")
model = ChatMistralAI(model="mistral-small")
parser = StrOutputParser()
chain = prompt | model | parser
result = chain.invoke({"topic": "ML"})
print(result)
```

→ Runnable Methods

Runnable provides three important methods:

↳ .invoke

Executes the runnable once for a single output

EX-

```
result = chain.invoke(input)
```

in short:- invoke = one input → one output

↳ batch()

processes multiple inputs together.

Ex

```
result = chain.batch(inputs)
```

in short:- chain.batch(inputs)

↳ stream()

Returns the output progressively instead of waiting for the complete response.

```
Token → Token → Token → ....
```

Useful for real time AI response.

In short: stream = progressive output

→ Runnable Parallel

It runs multiple independent Runnable pipelines simultaneously on the same input & returns their outputs as a dictionary.

Ex-

```
from dotenv import load_dotenv load_dotenv()
from langchain_mistralai import ChatMistralAI
from langchain_core.prompts import
ChatPromptTemplate
from langchain_core.output_parsers import
StrOutputParser
from langchain_core.runnables import
RunnableParallel

model = ChatMistralAI(model = "mistral-small-2506")
parser = StrOutputParser()
#Two different Prompts
short-prompt = ChatPromptTemplate.from_template
        ("Explain {topic} in 1-2 lines")
detailed-prompt = ChatPromptTemplate.from_template
        ("Explain {topic} in detail")
#Parallel Runnable
chain = RunnableParallel({
        "short": short-prompt | model | parser
        "detailed": detailed-prompt | model | parser})
result = chain.invoke({"topic": "ML"})
print ("SHORT EXPLANATION:")
print (result["short"])
print ("\n DETAILED EXPLANATION:")
Print (result["detailed"])
```

In short:- execute independent = RunnableParallel
Pipeline concurrently

→ **Runnable Lambda**

It converts a normal Python function/lambda into a Runnable, allowing you to add custom logic inside an LCEL pipeline.

Ex :-

```
#Input
topic = "Machine Learning"
chain = RunnableParallel({
  "short": RunnableLambda(lambda x: x
  ['short']) | short_prompt | model | parser,

  "detailed": RunnableLambda(lambda x: x
  ['detailed']) | detailed_prompt | model | parser
    })
result = chain.invoke({
  "short": {"topic": "Machine Learning"},
  "detailed": {"topic": "Deep Learning"}
    })
print (result['short'])
print (result['detailed'])
```

In short:- Runnable Lambda = custom Python logic inside an LCEL pipline

→ Runnable Passthrough

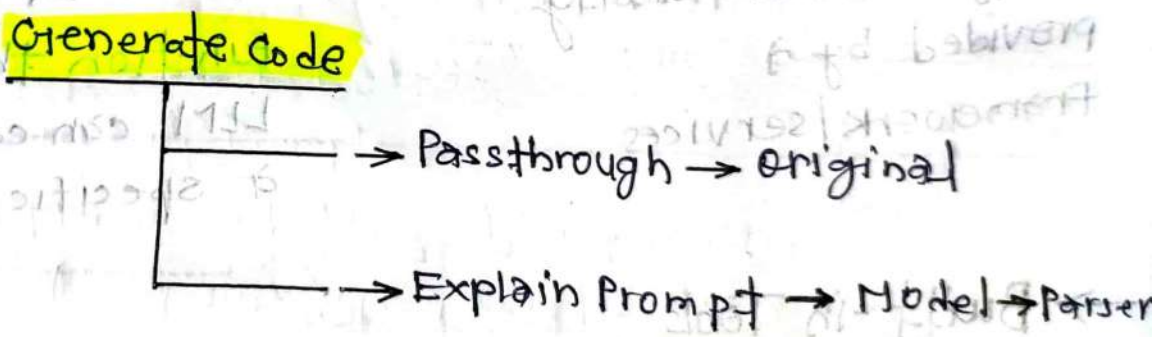
It passes the input unchanged to next step, allowing us to preserve data allowing us to preserve data while also processing it.

Ex-

```
from langchain_core.runnables import
RunnablePassthrough

explainer_chain = generate_code |
 RunnableParallel ( {
  "code" : RunnablePassthrough() ,
  "explanation" : explain_prompt | model |
   parser  y)
```

Flow



If we make generate_code | model | Parser | Explanation (output)

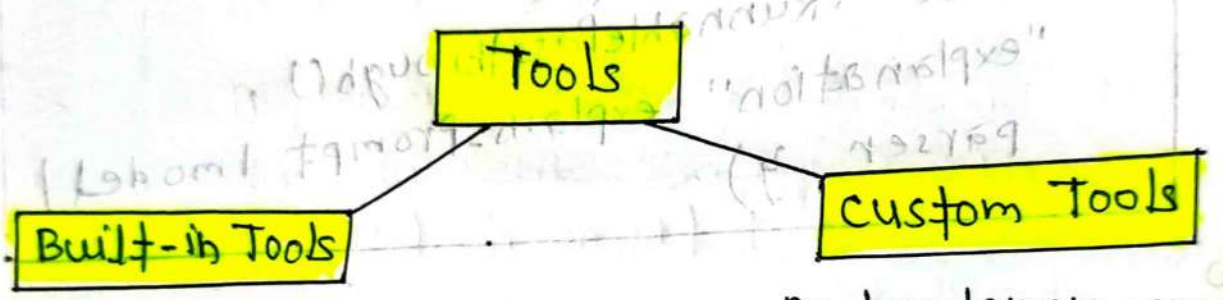
If we won't to see code generated i can't see because in pipeline we don't save generated code so i that situation we use Runnable pass through.

In short :- RunnablePassthrough = forward input unchanged while preserving it for the output.

→ Tools

It is simply a function or external service that an LLM can use to do real-world tasks.

Ex- • Web search
- Calculator
- Database
- API
- Python function



- Readymade functionality provided by a framework/services

- A developer-create function that an LLM can call to perform a specific task.

→ Built-in Tools

- Langchain provides pre-built tools for common tasks. As a developer we don't need to build everything from scratch.

↳ Ex: • Tavily search
- DuckDuckGo search
- Google search
- Calculator

↳ Tavily Search

It is a web search tool designed for AI applications / agent.

It Provides access to current web Information.

• Implemention Code

```
from langchain_community.tools.tavily_search
import TavilySearchResults

search_tool = TavilySearchResult(max_result = 5)

result = search_tools.invoke (
        "Latest AI news of 2026")

print(result)
```

Flow

Question → Tavily Tools → Web search → Results

→ Custom Tools

When built-in tools aren't enough, we can create our own tool.

↳ Useful for:

- custom calculations
- Database queries
- Private API's
- Business Logic
- Internal systems

Three requirements

(1) @tool :- It converts a Python function into a Langchain tool.

(2) Type hint :- Define input / output types

```
text : str -> int
```

(3) Docstring :- It explains the tool's purpose to the LLM

Code Implementation

```
from langchain.tools import tools

@tool #Decorater for creating a tool
def get_greeting(name:str) ->str:
#type hints
    """Generate a greeting message for a user"""
#docstring
    return f"Hello {name}, Welcome to AI
    world"

# Execute the custom tool
result = get_greeting.invoke({"name": "Aniket"})
print(result)
```

→ Tool Binding

connecting one or more tools to an LLM so the LLM knows which tools are available & how to call them

```
llm_with_tools = llm.bind_tools([
    get_text_length])
```

↳ Flow:

```
Tool → bind_tools() → LLM
```

↳ Ex

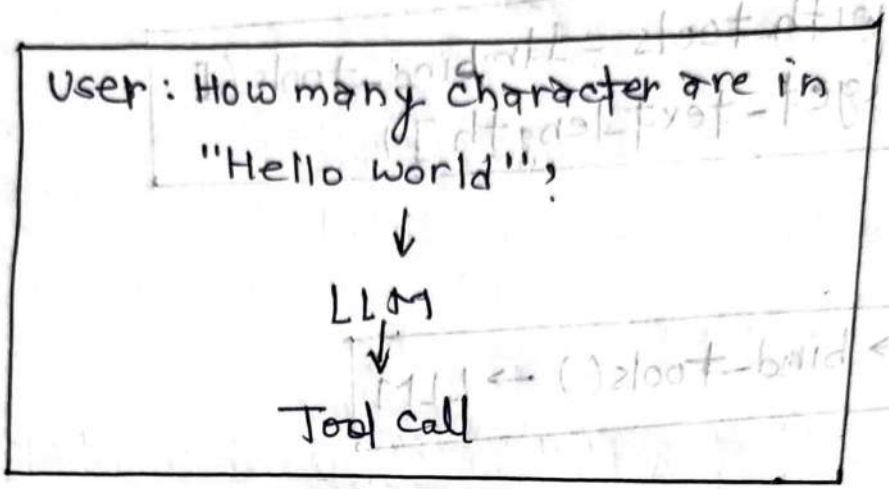
```
from dotenv import load_dotenv
load_dotenv()
from langchain_mistralai import ChatMistralAI
from langchain.tools import tool
@tool
def get_text_length(text: str) →
int:
    """Return the number of characters
in a text. """
    return len(text)
llm = ChatMistralAI (model ="mistral-small-2506")
llm_with_tools =llm.bind_tools ([get_text_length])
```

In short:- Tool Binding = make tool available

→ Tool calling

the LLM analyzes the user's request & generates a structured request to use one of its bound tools. short - LLM requests tool

↳ Ex



Note:- The LLM does not execute the function It only says

Call "get-text-length" with this input.

↳ Ex code

```
from langchain_core.messages import HumanMessage
messages = [HumanMessage(
    content = 'How many characters are in "Hello
              word" ? ' )]
response = llm_with_tools.invoke(messages)
print (response.tool_calling)
```

↳ In output these terms appears

- name - Tool to call
- args - Input Arguments
- id - Unique ID of the call

→ Tool Execution

It is the application receives the LLM's tool call & actually runs the requested function with the provided arguments.

short- Here code actually runs tool

↳ Flow

LLM → Tool call → Application → Tool Execution

↓

Tool Result

↳ Ex code

```
tool_call = response.tool_calls[0]
result = get_text_length.invoke(tool_call["args"])
print(result)
```

→ Tool Message

After execution, the tool result needs to be returned to the LLM.

"ToolMessage" represents the result of a tool execution.

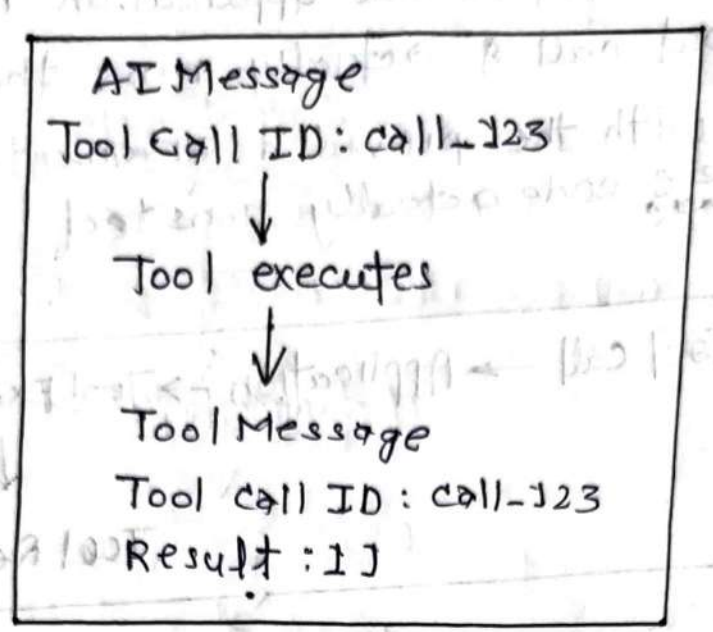
↳ Ex

```
from langchain_core.message import ToolMessage

tool_message = ToolMessage(
    content=str(result),
    tool_call_id = tool_call["id"])
```

• tool_call_id = It connects the result to the exact tool call

ex-



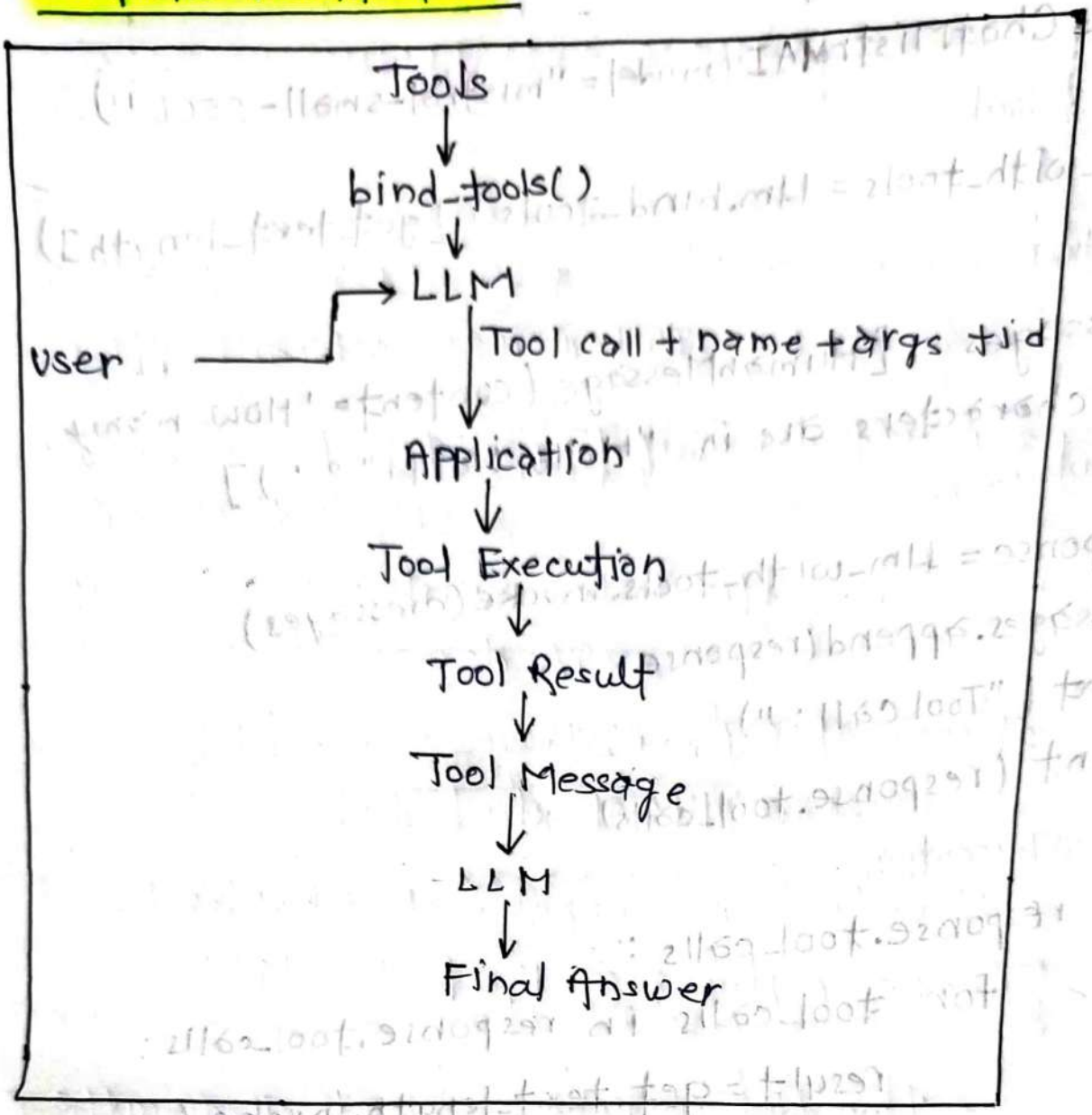
→ Complete code Implementation:

```
from dotenv import load_dotenv
load_dotenv()
from langchain_mistralai import ChatMistralAI
from langchain.tools import tool
from langchain_core.messages import HumanMessage,
 ToolMessage
 # create Tool
 @tool
 def get_text_length(text: str) ->
 int:
     """ Return the number of character in a
 text """
     return len(text)
```

```
#create LLM
llm = ChatMistralAI(model="mistral-small-2506")
#Bind Tool
llm_with_tools = llm.bind_tools([get_text_length])
#user Message
messages = [HumanMessage(content='How many
    characters are in "Hello World"?')]
#Tool calling
response = llm_with_tools.invoke(messages)
messages.append(response
print("Tool call:")
print(response.tool_calls)
#Tool Execution
if response.tool_calls:
        for tool_calls in response.tool_calls:
            result = get_text_length.invoke(
            tool_call["args"])
            print("Tool Result:")
            print(result)

#Return Result to LLM
messages.append(ToolMessage(content=str(result),
  tool_call_id=tool_call["id"]))
#Final LLM Response
final_response = llm_with_tools.invoke(messages)
print("Final Answer:")
print(final_response.content)
```

→ Complete Architecture



→ Multiple Tools

If we have multiple tools:

```
@tool
def add(a: int , b : int) -> int:
    """ Add two numbers. """
    return a+b

@tool
def multiply (a: int, b: int) -> int:
    """ Multiply two numbers. """
    return a*b
```

Bind Both:

```
llm_with_tools = llm.bind_tools([add, multiply])
```

then the LLM can then request whichever tool is appropriate.

→ Dynamic Tool Execution

for multiple tools, create a tool dictionary:

```
tools = {"add": add,
         "multiply": multiply}
```

Then execute dynamically:

```
for tool_call in response.tool_calls:
    tool_name = tool_call["name"]
    tool_args = tool_call["args"]
    selected_tool = tools[tool_name]
    result = selected_tool.invoke(tool_args)
    print(result)
```

It is the basic mechanism behind a tool execution loop.

PROJECT : 1
(CITY INTELLIGENCE AGENT)

→ **Overview**

- Its a AI-powered city Intelligence agent built with Langchain + Mistral AI.
- **Weather Tool** : Fetches real-time weather using OpenWeather API.
- **News Tool** : Retrieves latest city news using Tavily search API.
- **AI agent** : uses create_agent() to decide which tools to call
- **Tool calling & Execution** : Demonstrates the complete LLM → tool call → execution → result → LLM flow
- **Human in the loop** : @wrap_tool_call asks for approval before executing tools.
- **Enviroment security** : API keys managed through .env using python-dotenv.
- **Interface** : we use streamlit for UI.
- **Development Approach** :

(1) Human-in-the loop + AI Assisted (Human + AI)

(2) Automation / heavy AI

(1) Project Goal

It's a assistant that can:

- Get live weather.
- search latest city news
- Let the LLM decide which tool to use
- Ask human approval before tool execution

(2) Build Custom Tools

Weather Tool

city → OpenWeather API → JSON → Weather Result

- created get_weather() & exposed it using @tool.
- Added API error handling & formatted the response.

News Tool

city → Tavily search → Results → Formatted News

- Created get_news() using Tavily.
- Extracted title, URL & content

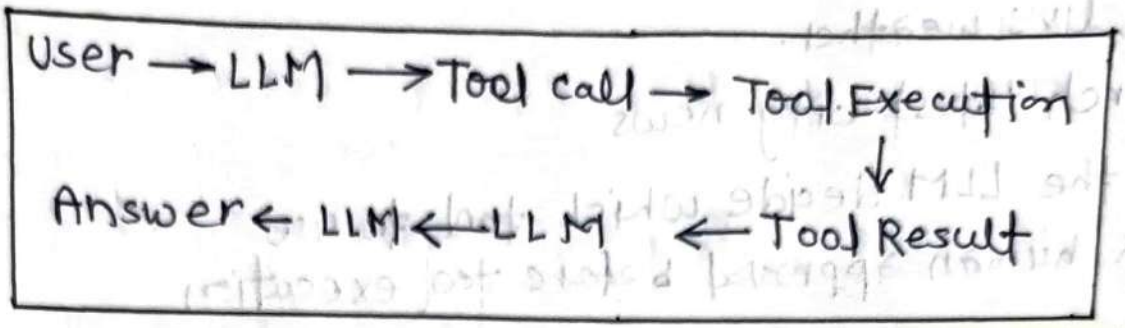
(3) Connect LLM

```
llm = ChatMistralAI (model = "mistral-small-2506")
```

- Mistral acts as the reasoning / decision layer.
- LLM decides when a tool is required.

(4) Understand Manual Agent Loop

Flow:



Manual loop provide control & visibility but create:

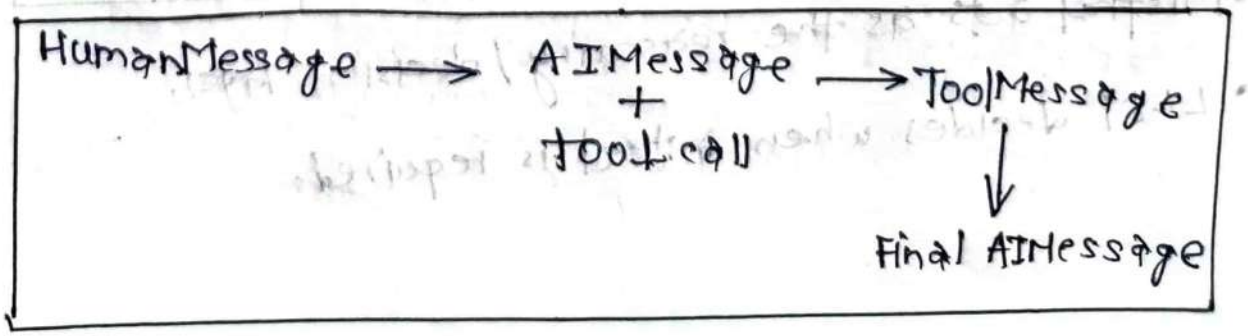
- More boilerplate
- Difficult state management
- Scaling & maintenance problems.

(5) Use "create_agent"

```
agent = create_agent (llm,
        tools = [get_weather, get_news],
        system_prompt = "You are a helpful
        city assistant.")
```

- Replaces most manual tool execution logic.
- Automatically manages the agent/tool loop & message state

Message Flow

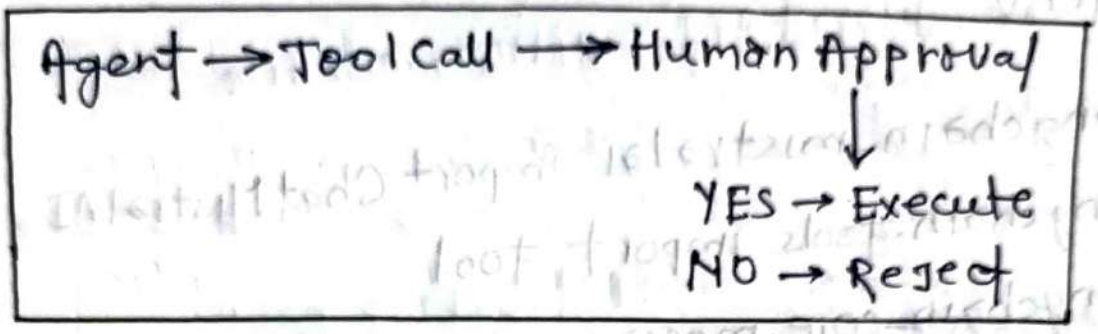


· Final Answer:

```
result["messages"][-1].content
```

↳ Add Human-in-the-Loop

Used @wrap_tool_call to intercept tool execution.

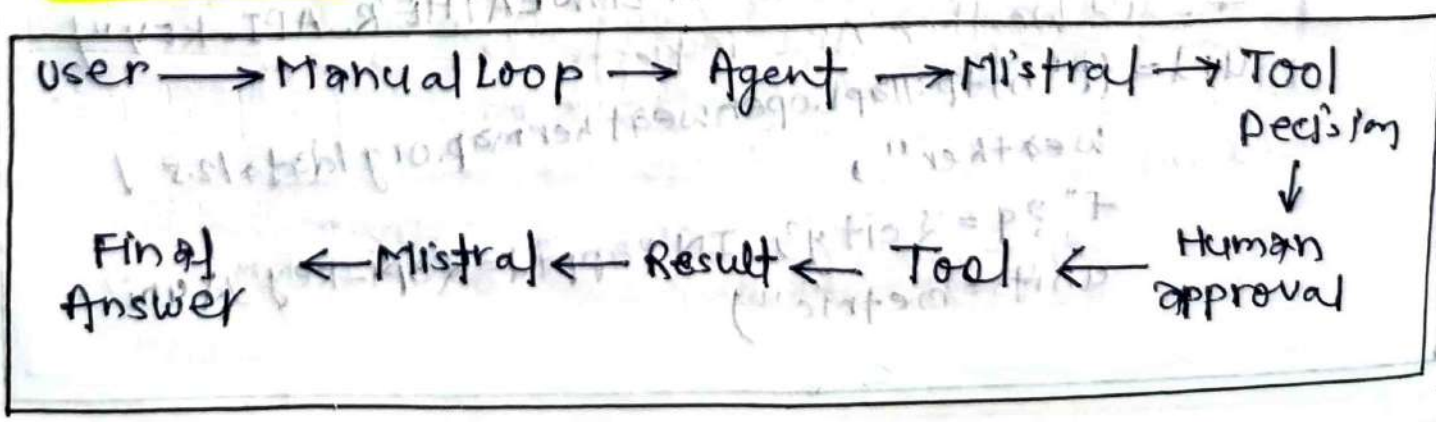


- handler(request) - executes approval tool
- ToolMessage - safely returns a denial when rejected.

↳ components

- LLM - Mistral
- Agent - Langchain create_agent
- Tools - OpenWeather + Tavily
- Middleware - Human approval
- Interface - Manual python while loop

↳ core Flow



Complete Code Implementation Automation Approach

```
# 1. IMPORT & ENVIROMENT
from dotenv import load_dotenv
import os
import requests

load_dotenv #Load API key from .env file

from langchain_mistralai import ChatMistralAI
from langchain.tools import tool
from langchain_core.messages import ToolMessage
from tavily import TavilyClient
from rich import print
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_tool_call

# 2. WEATHER TOOL
@tool
def get_weather(city: str) -> str:
    """Get current weather of a city"""
    # Get OpenWeather API key
    api_key = os.getenv("OPENWEATHER_API_KEY")
    # Build weather API requests
    url = (f"http://api.openweathermap.org/data/2.5/
           weather"
           f"?q={city},IN&appid={api_key}&
           units=metric")
```

```
    #Call weather API
    response = requests.get(url)
    data = response.json()
    # Handle API errors
    if str(data.get("cod")) != "200" :
        return f"Error: {data.get('message',
        'could not fetch weather')}"
    #Extract useful weather information
    temp = data["main"]["temp"]
    desc = data["weather"][0]["description"]

    return f"Weather in {city}: {desc}, {temp}°C"
```

==========

#3. NEWS TOOL

==========

```
#create Tavily search client
tavily_client = TavilyClient(api_key=os.getenv(
                "TAVILY_API_KEY"))
@tool
def get_news(city: str) -> str:
    """Get latest news about a city"""
    #search the web for latest city news
    response = tavily_client.search(
        query=f"latest news in {city}",
        max_result=3)
    result = response.get("results", [])
    #handle empty search result
    if not results:
        return f"No news found for {city}"

    news_list = []
```

```
    # Extract relevant information from results
    for r in results:
        title = r.get("title", "No title")
        url = r.get("url", "")
        snippet = r.get("content", "")

        news_list.append(
            f"- {title}\n"
            f"  {url}\n"
            f"  {snippet[:100]}..."
        )
    return (
        f"Latest news in {city}:\n\n"
        + "\n\n".join(news_list)
    )
```

```
# ==========
# 4. LLM SETUP
# ==========

# Initialize the Mistral LLM
llm = ChatMistralAI(model="mistral-small-2506")

# ==========
# 5. HUMAN-IN-THE-LOOP
# ==========

@wrap_tool_call
def human_approval(request, handler):
    """Ask the user before executing every tool."""
    # Identify which tool the agent want to see
    tool_name = request.tool_call["name"]
    # Ask the human for permission
    confirm = input(f"Agent wants to call '{tool_name}'. "
                    "Approve? (yes/no): "
                    )
```

```
        #Reject the tool call if user says no
        if confirm.lower() != "yes":
            return ToolMessage(
                content = "Tool Call denied by user. ",
                tool_call_id = request.tool_call["id"]
            )
        #Execute the tool if approved
        return handler(request)
```

#6. CREATE AI AGENT

```
agent = create_agent(llm,
        #Give the agent access to these tools
        tools = [get_weather, get_news],
        #Define the agent's behaviour
        system_prompt = "you are a helpful city assistant.",
        #Add human approval middleware
        middleware = [human_approval]
)
```

#7. MANUAL AGENT LOOP

```
print("City Agent | type exit to quit")
while True:
        #Take input directly from the user
        user_input = input("You: ")
        #stop the Program
        if user_input.lower() == "exit":
            break
        #send user request to the agent
        result = agent.invoke({"messages": [{
                "role": "user"
                "content": user_input }]})
```

```
#Display the agent's final response
print( "bot: ",
        result ["messages"][-1].content )
```

→ **Manual Agent Loop**

It is an agent workflow where the developer manually controls the entire LLM → tool call → tool execution → result → LLM cycle using code

- This Manual approach gives full visibility & control over every step. we can see
  - when LLM is called
  - when tool is triggered
  - what data is passed
  - what result comes back
  - How the result returns to the LLM
  - when the loop stops.

↳ **Problems as the project grows**

(1) **Too much boilerplate**

↳ repetitive setup code

```
Loop + Tool Logic + Messages + conditions
```

(2) **Error-Prone**

A small mistake in conditions, continue or termination logic can break the agent.

(3) Difficult to scale

2 tools → manageable
10+ tools → Complex
Multiple workflows → Messy

(4) Maintenance overhead

More tools + features + logic = harder to maintain.

→ create_agent

It is a high level langchain abstraction that handles the repetitive agent execution logic internally.

Instead of manually managing:

Loop → Tool call → Tool execution → Message handling
↓
stop handling

we provide: ex

```
agent = create_agent (llm,
        tools = [get_weather, get_news])
```

- These stays same like Manual Agent Loop

LLM decides → Tool executes → External data returned → LLM processes result → Final Answer

→ Manual Agent Vs create_agent

Manual Agent	create_agent
(1) Full control	(1) Higher Level abstraction
(2) Full visibility	(2) Hides internal complexity
(3) More code	(3) Less boilerplate
(4) Manual state / loop management	(4) Framework-managed
(5) Harder to scale	(5) Easier to scale

→ code Implementation of Human-in-the-loop + AI

Till (3) step - NEWS TOOL
everything will be same but after this we use Human-in-the-loop & Manual Agent.

```
# #4. LLM SETUP
llm = ChatMistralAI(model="mistral-small-2506")
#create tool registry for manual lookup
tools = { "get_weather": get_weather,
          "get_news": get_news }
#Binding tools to the LLM so it can generate tool calls
llm_with_tools = llm.bind_tools(
    [get_weather, get_news])
```

```
# AGENT LOOP
# store conversation & tool-call Messages
messages = [ ]
print("\n City Intelligence Agent")
print ("Type 'exit' to quit\n")

#outer loop handles user interaction
while True:
    user_input = input("you: ")
    if user_input.lower() == "exit" :
        break
    #Add user message to conversation
    messages.append(HumanMessage (content = user_input)
                    )
    #Inner Loop handles LLM ↔ Tool execution
    whileTrue:
        #Ask LLM to process current message state
        result = llm_with_tools.invoke(messages)
        #store LLM response
        messages.append(result)
        #if tool is required
        if result.tool_calls :
            #Process every requested tool call
            for tool_call in result.tool_calls :
                tool_name = tool_call ["name"]
                # HUMAN-IN-THE-LOOP
                #Ask human before executing the tool
                confirm = input (
                  f"\n Agent wants to call '{tool_name}' "
                  "Approve ? (yes/no): "
                  )
```

```
                #Reject the tool call
                if confirm.lower() != "yes":
                    print("Tool call denied\n")
                    break
                #Execute tool
                tool_result = tools[tool_name].invoke(
                tool_call)
                #IMPORTANT: use ToolMessage
                #send tools result back into message
                  history
                messages.append(ToolMessage(
                    content = tool_result,
                    tool_call_id = tool_call["id"]))
            # Continue the loop so LLM can process
              tool result
            continue
        #Final Answer
        else:
            # No tool call -> LLM has produced final
              response
            print("\nFinal Answer:\n")
            print(result.content)
            print("\n" + "=" * 50 + "\n")
            break
```

Here we need to say or type "yes" to execute but in create_agent it will do automatically